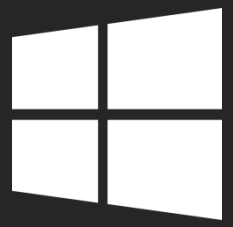
Требования, предъявляемые к современным ОС

Требования, предъявляемые к современным ОС

Главное требование - выполнение основных функций эффективного управления ресурсами и обеспечение удобного интерфейса для пользователя и прикладных программ.

1. прозрачность (незаметность) работы служебных программ
2. гарантированная надежность
3. максимальная скорость выполнения
4. минимальный машинный код
5. использование стандартных средств для связи с проблемными программами

Сравнение количества строк кода в ОС Linux и Windows



Год	Версия	Строк кода
1993	Windows NT 3.1	5 600 000
1995	Windows NT 3.5	8 400 000
1996	Windows NT 4.0	16 000 000
2000	Windows 2000	29 000 000
2001	Windows XP	35 000 000
2007	Windows Vista	45 000 000
2009	Windows 7	42 000 000
2012	Windows 8	50 000 000
2015	Windows 10	55 000 000



Android 12 млн. строк

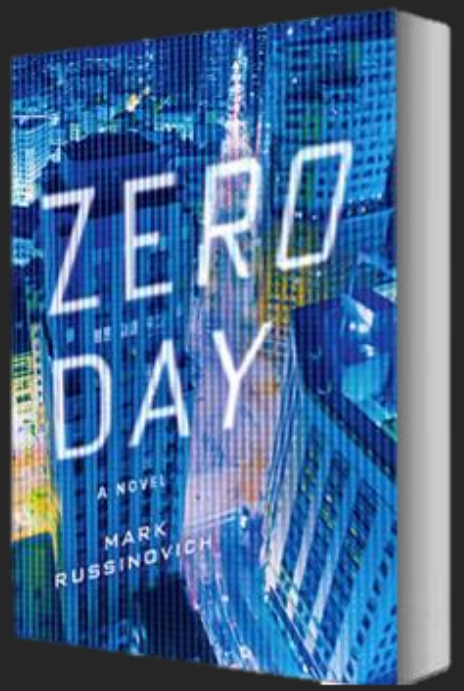


Год	Версия	Строк кода
1991	Ядро Linux 0.1	10 000
1994	Ядро Linux 1.0.0	176 000
1995	Ядро Linux 1.2.0	311 000
1996	Ядро Linux 2.0.0	778 000
1999	Ядро Linux 2.2.0	1 801 000
2001	Ядро Linux 2.4.0	3 338 000
2003	Ядро Linux 2.6.0	6 000 000
2009	Ядро Linux 2.6.32	12 607 000
2017	Ядро Linux 4.11.7	18 500 000
2017	Ядро Linux 4.12	24 100 000
2018	Ядро Linux 4.18	25 300 000
2019	Ядро Linux 5.2	26 550 000
2020	Ядро Linux 5.8	28 360 000

Дистрибутивы Linux «тяжелее». Больше всего строк в Debian, который имеет около 419 миллионов строк кода (почти в 4 раза больше, чем у Red Hat).

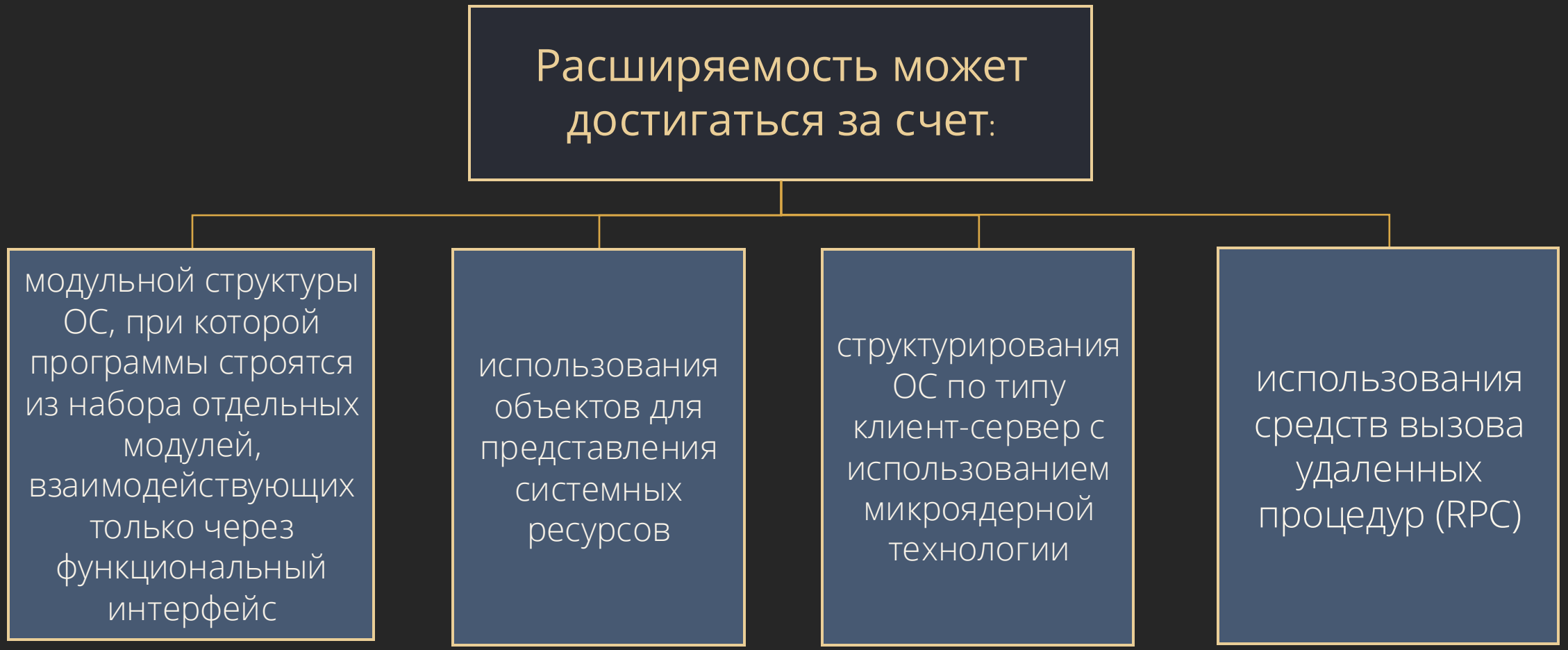
Уязвимость нулевого дня

Zero-day эксплойт — киберугроза, использующая ошибку или уязвимость в приложении или операционной системе и появившаяся сразу после обнаружения данной уязвимости, пока разработчики ПО еще не успели создать патч, а IT-администраторы — принять другие меры безопасности. Происхождение термина связано с тем, что уязвимость или атака становится публично известна до момента выпуска производителем ПО исправлений ошибки (то есть потенциально уязвимость может эксплуатироваться на работающих копиях приложения без возможности защититься от неё).



Расширяемость

Код должен быть написан так, чтобы систему можно было легко наращивать и модифицировать по мере изменения потребностей рынка.



Переносимость

Переносимость (многоплатформенность) дает возможность перемещать всю систему на машину, базирующуюся на другом процессоре или аппаратной платформе, делая при этом по возможности минимальные изменения в коде.



1

использовать язык высокого уровня



2

изолировать процессор

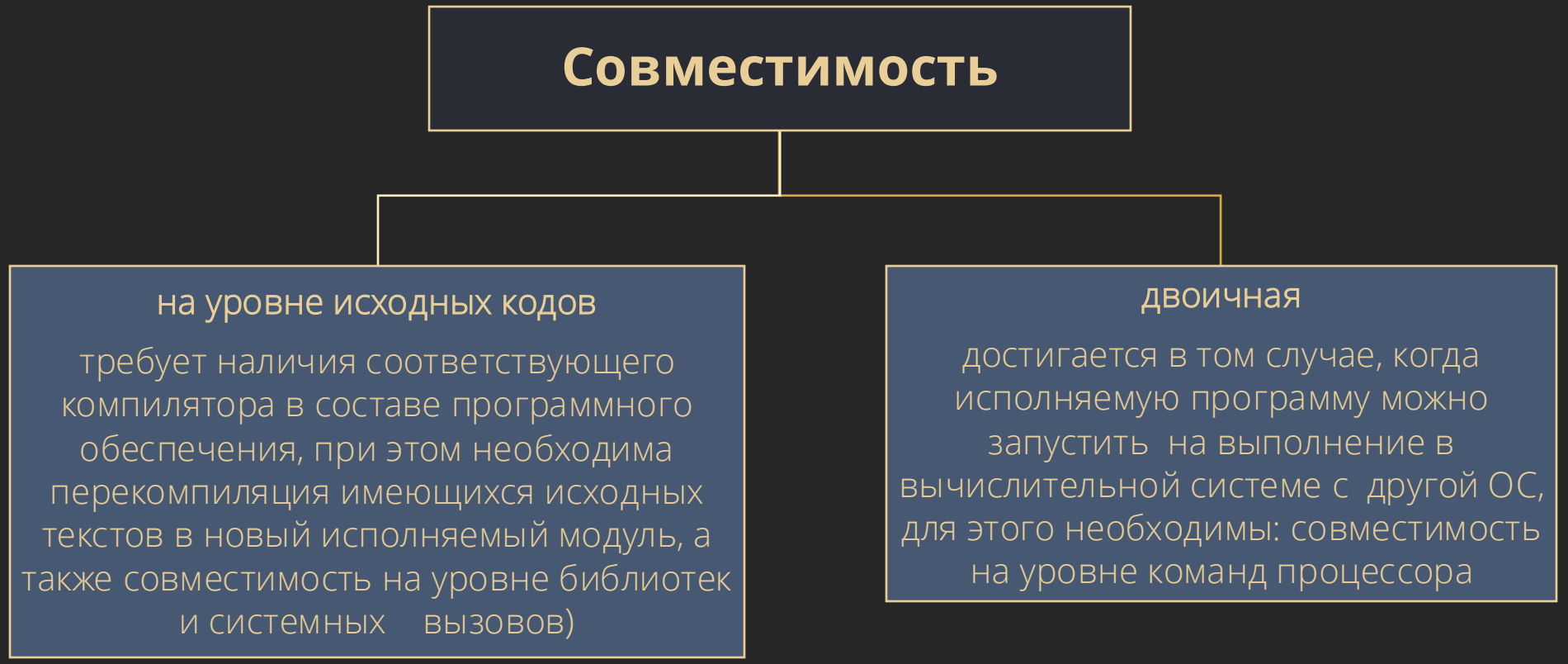


3

изолировать платформу

Совместимость

Совместимость - способность ОС выполнять программы, написанные для других ОС или для более ранних версий данной операционной системы, а также для другой аппаратной платформы.



При совпадении архитектур процессоров (набора команд и диапазона адресов) двоичная совместимость достигается при:

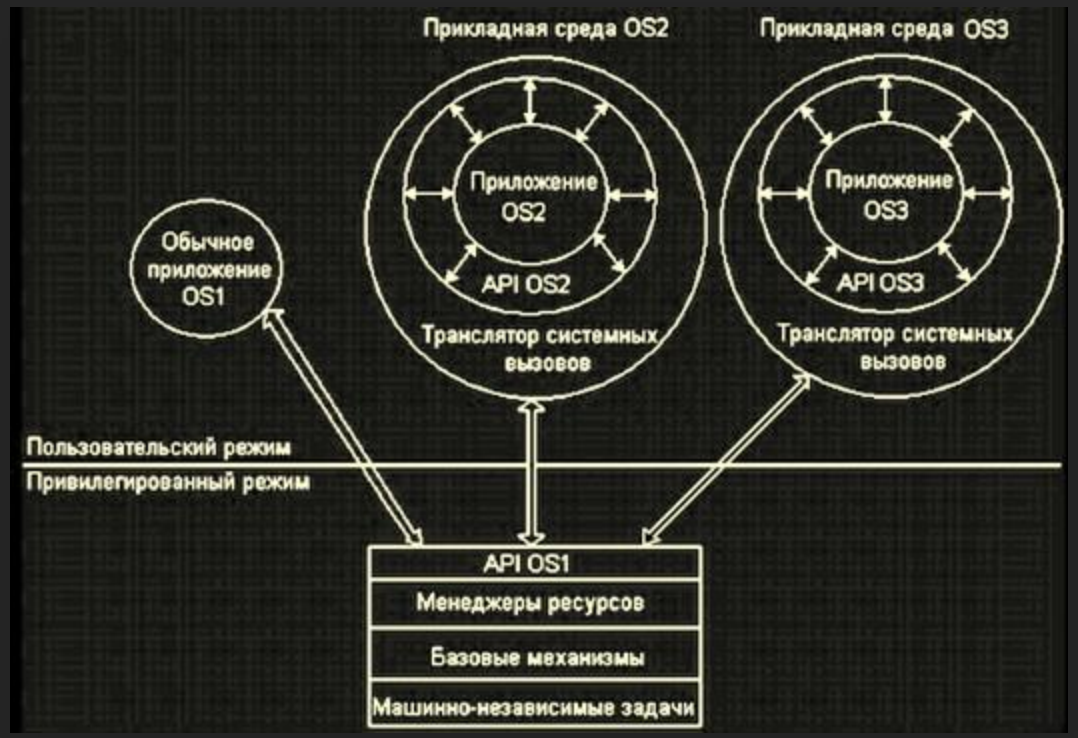
1. поддержке вызовов API-функций новой ОС
2. соответствии внутренней структуры исполняемого файла правилам новой ОС

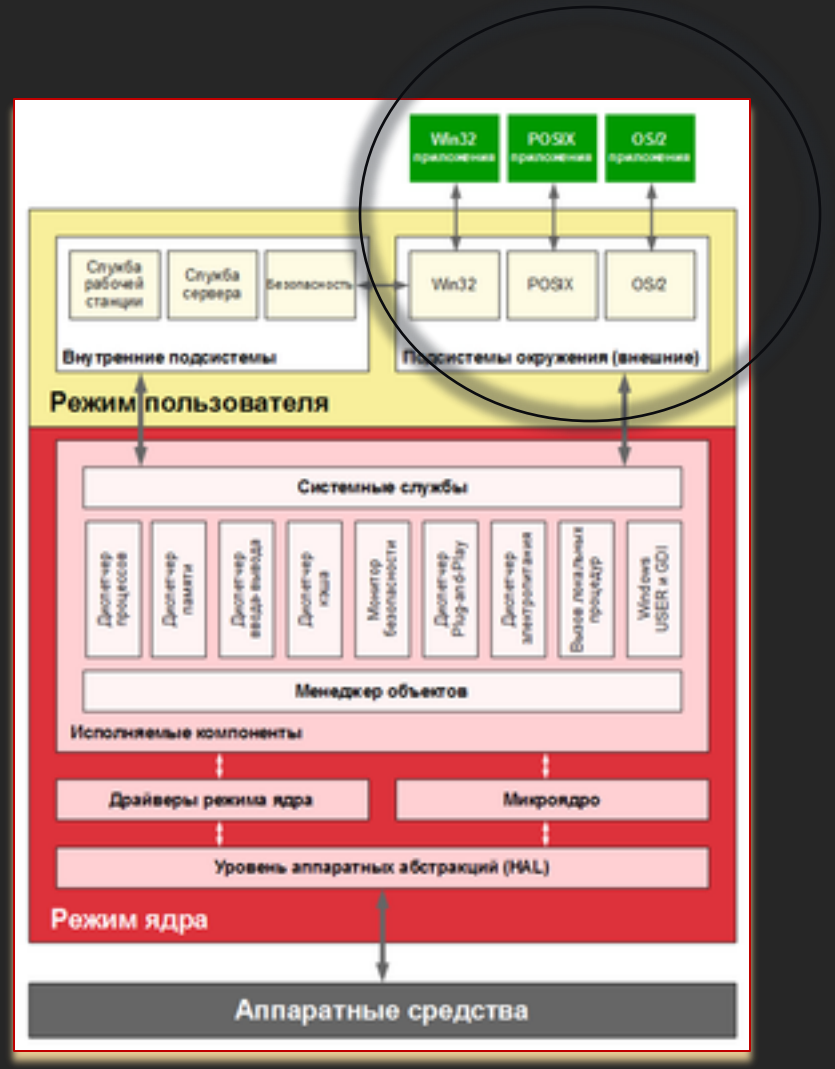
Способы достижения двоичной совместимости в случае различных архитектур

1. эмуляция
2. использование множественных прикладных программных сред
3. система виртуальных машин (СВМ)

WSL1, WSL2, Wine ...

Множественные прикладные программные среды





Windows 2000

Виртуализация

Система виртуальных машин



Вариант организации вычислительного процесса, при котором на одном компьютере одновременно выполняются несколько копий одной и той же или нескольких разных ОС. Каждая из этих ОС функционирует так же, как если бы она выполнялась на отдельном компьютере.



Виртуальная машина — программная и/или аппаратная система, эмулирующа
аппаратное обеспечение некоторой платформы* и исполняющая програм
для нее на host-платформе ** или виртуализирующая некоторую платф
создающая на ней среды, изолирующие друг от друга программы и даже
операционные системы.

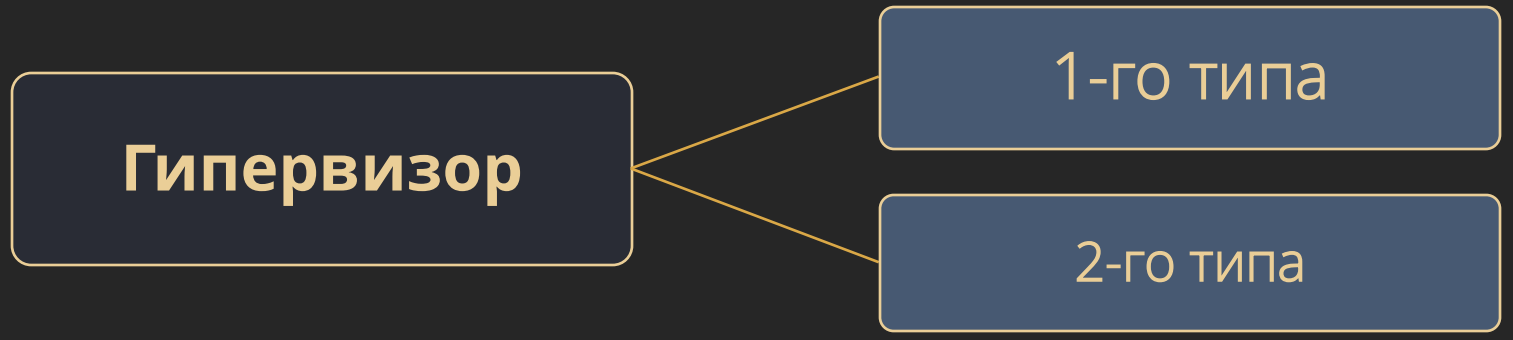


Помимо процессора, ВМ может эмулировать работу как отдельных компонентов аппаратного обеспечения, так и целого реального компьютера (включая BIOS, оперативную память, жёсткий диск и другие периферийные устройства).

*target — целевая, или гостевая платформа
** host — хост-платформа, платформа-хозяин



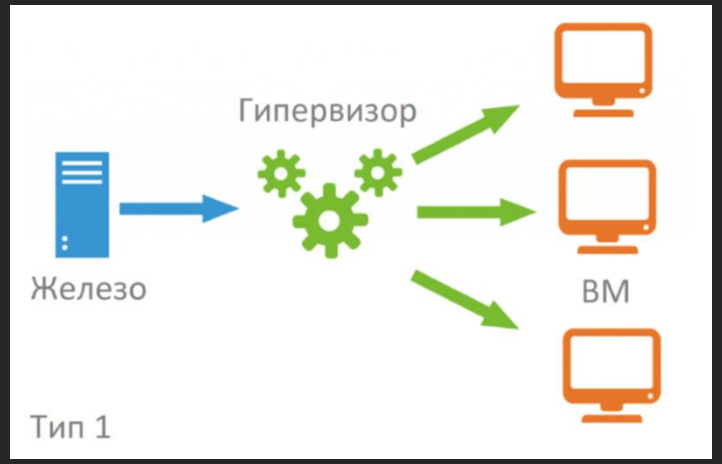
Основная идея: **монитор виртуальных машин** (Virtual Machine Monitor (**VMM**)) создает иллюзию присутствия нескольких (виртуальных) машин на одном и том же физическом оборудовании. VMM известен также как **гипервизор**.

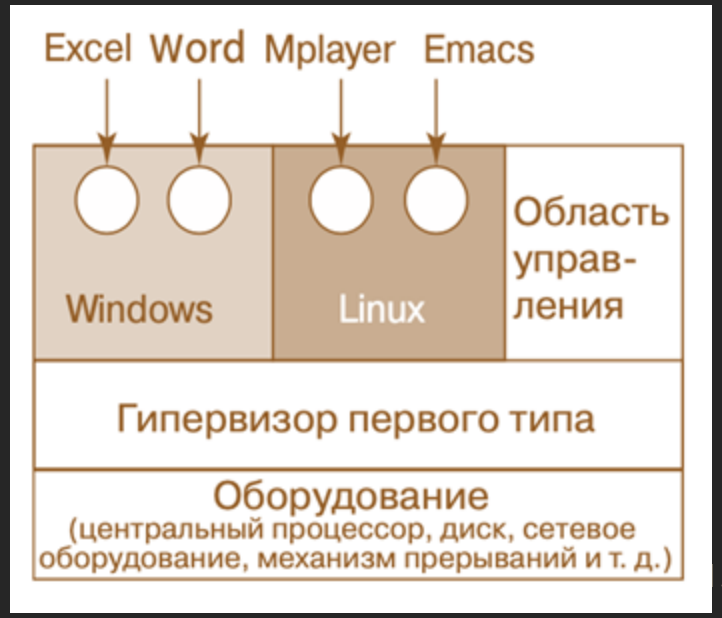


Гипервизор первого типа
автономный гипервизор

Запускается на аппаратном обеспечении хоста, управляет оборудованием и виртуальными машинами. Этот гипервизор похож на ОС, поскольку это единственная программа, запущенная в самом привилегированном режиме.

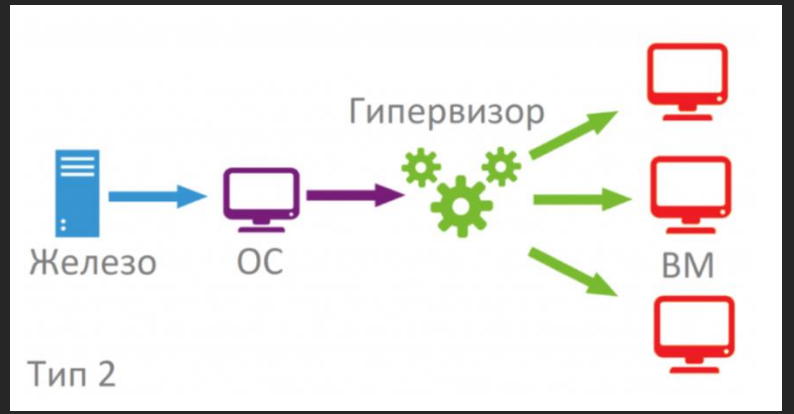
К этому типу относятся гипервизоры:
VMware ESX / ESXi
Microsoft Hyper-V
Xen, Oracle VM Server для SPARC
Oracle VM Server для архитектуры x86



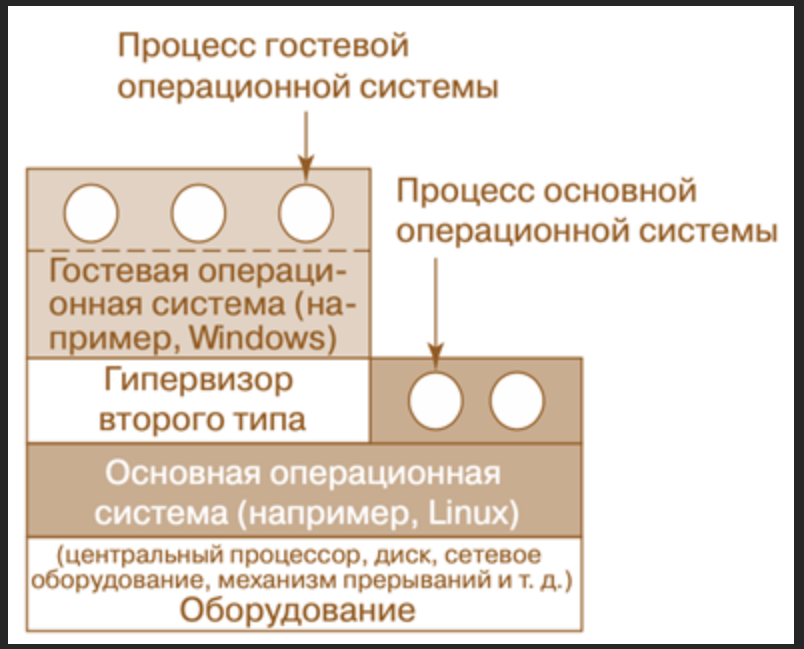


Гипервизор второго типа
хостовый гипервизор

Запускается как процесс пользовательского режима. Разделяет ОС машины хоста и ОС гостевой машины, а гостевая ОС представлена обычным процессом хоста.

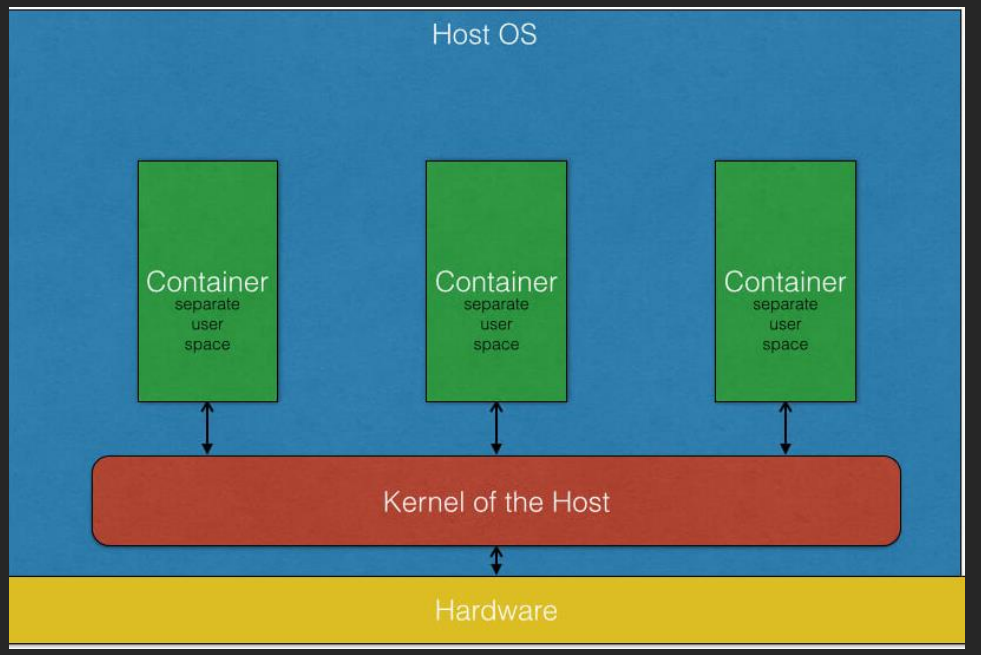


К этому типу гипервизоров относятся:
VMware Workstation
VMware Player
VirtualBox
Parallels Desktop для Mac.



Контейнеры

Контейнеризация – это легковесная виртуализация и изоляция ресурсов на уровне операционной системы, которая позволяет запускать приложение и необходимый ему минимум системных библиотек в полностью стандартизованном контейнере при помощи определенных интерфейсов.
Контейнер не зависит от ресурсов или архитектуры хоста, на котором он работает.
Все компоненты, необходимые для запуска приложения, упаковываются как один образ и могут быть использованы повторно. Приложение в контейнере работает в изолированной среде и не использует память, процессор или диск хостовой операционной системы. Это гарантирует изолированность процессов внутри контейнера



Реализация

- В основе — изоляция процессов через *cgroups, namespaces* и другие механизмы
- Используется только одно ядро хостовой ОС
- Возможны ограничения ресурсов по CPU, RAM, сети и т.д.
- Реализации:
  - Docker
  - LXC
  - OpenVZ (Virtuozzo)

Средства изоляции

Пространство имён (*namespaces*):

- **PID** PID процессов
- **NETWORK** Сетевые устройства, стеки, порты и т.п.
- **USER** ID пользователей и групп
- **MOUNT** Точки монтирования
- **IPC System V** IPC, очереди сообщений POSIX
- **UTS** Имя хоста и доменное имя NIS

Контрольные группы (*cgroups*)

ограничение ресурсов:

- использование памяти, в том числе виртуальной
- приоритезация: разным группам можно выделить разное количество процессорного ресурса
- пропускной способности подсистемы ввода-вывода

Контейнеры

Скорость создания

Экономичность

Контейнер занимает меньше места в хранилище.

Высокая производительность разработки Отсутствие межсетевых зависимостей и конфликтов. Каждый контейнер представляет собой микросервис, который можно независимо обновлять.

Управление версиями

Можно мониторить версионность контейнеров, следить за различиями между ними.

Возможность миграции среды вычислений

Стандартизация

Как правило, контейнеры создаются на основе открытых стандартов. Поэтому с ними можно работать в большинстве дистрибутивов Linux, Microsoft, MacOS. Все зависимости приложений и ОС, необходимые для работы приложения, инкапсулируются. Это позволяет без труда переносить образ контейнера из одной среды в другую.

Безопасность

Контейнеры изолированы друг от друга и базовой инфраструктуры. Изменение/обновление/удаление одного контейнера не влияет на другой.

Высокая сложность

Рост количества контейнеров, работающих с приложением, влияет на сложность управления ими. В производственной среде для работы с множеством контейнеров стоит использовать оркестраторы. Например, Kubernetes и Mesos.

Разрастание

Нередко в контейнеры упаковывается гораздо больше ресурсов, чем реально требуется. Из-за этого образ разрастается, занимая больше места на диске.

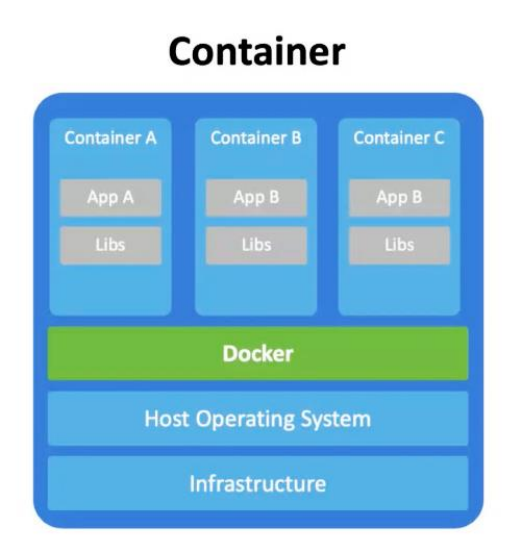
Поддержка Native Linux

Docker и многие другие контейнерные технологии основаны на Linux-контейнерах (LXC). Из-за этого запуск контейнеров в Windows-среде не всегда удобен, а их использование сложнее, чем при работе в Linux.

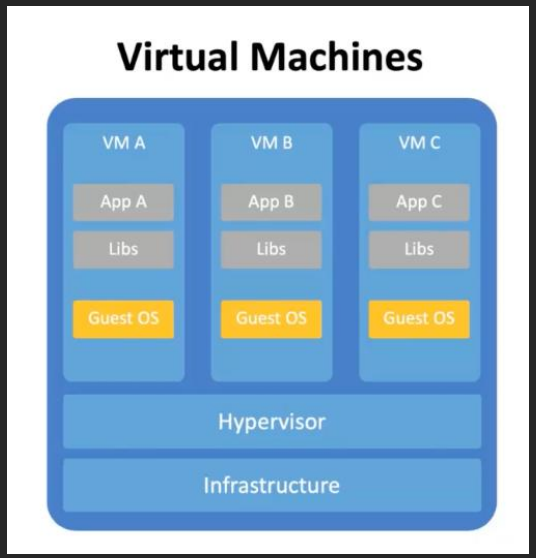
Недостаточная зрелость

Технологии контейнеризации приложений появились на рынке сравнительно недавно. Не всегда удаётся сразу решить возникшую проблемы. Иногда требуется время на поиск решения.

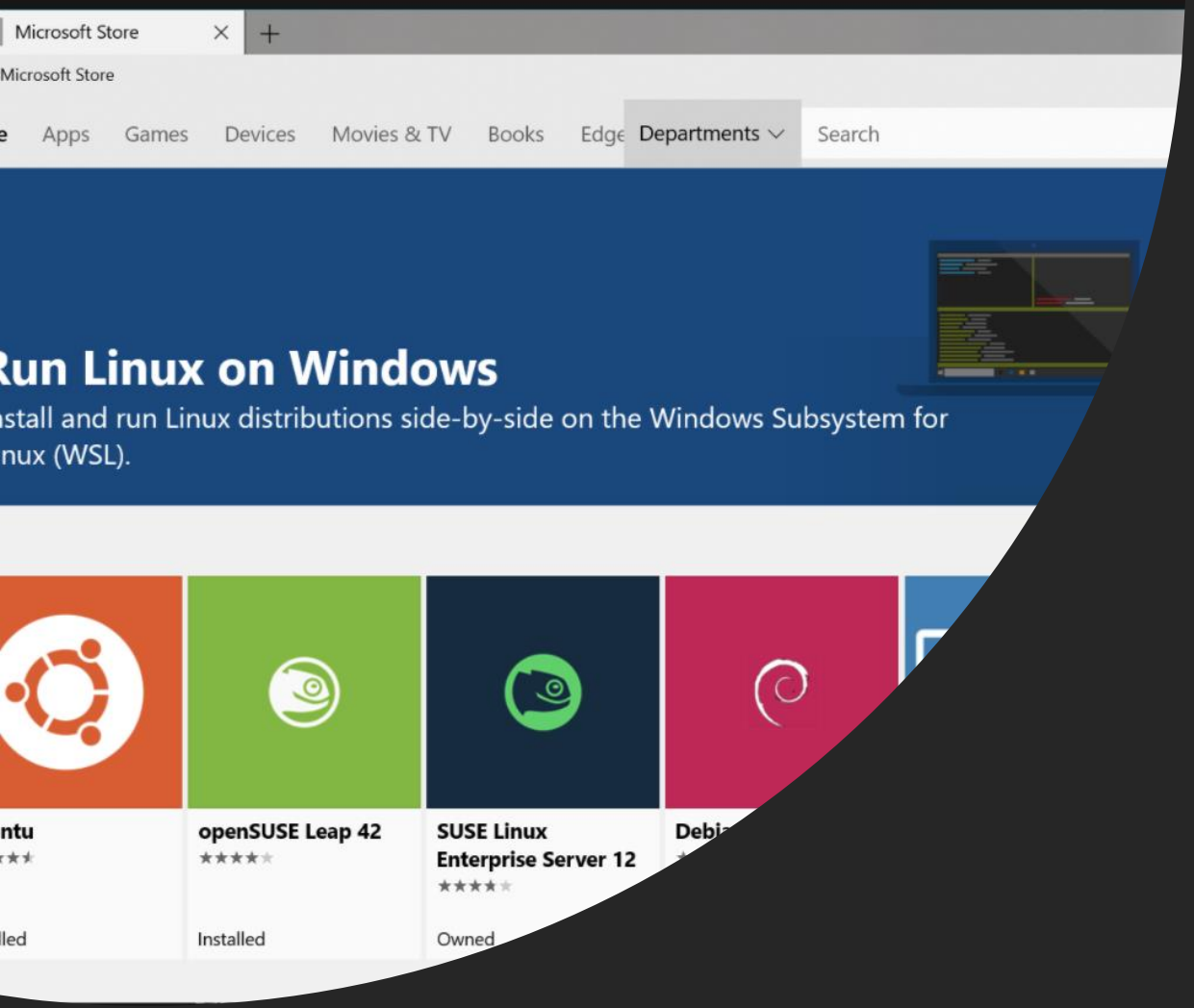
Контейнеры VS виртуальные машины



- одно ядро (хостовое)
- изоляция на уровне окружения (библиотеки, утилиты, конфигурации)
- быстрый запуск
- низкие накладные расходы



- полная свобода выбора ядер, модулей ядра
- эмуляция железа (сетевые устройства, контроллеры и т.д.)
- разнородные ОС (Windows, Linux, FreeBSD) на одном хосте
- полная изоляция гостевых систем
- высокие накладные расходы



WSL

Подсистема Windows для Linux позволяет разработчикам запускать среду GNU/Linux с большинством программ командной строки, служебных программ и приложений непосредственно в Windows без каких-либо изменений и необходимости использовать традиционную виртуальную машину или двойную загрузку.

https://docs.microsoft.com/ru-ru/windows/wsl/install-win10

https://m.habr.com/ru/company/vdsina/blog/535214/

WSL1

Подсистема Linux в ядре NT — это реализация уровня системных вызовов Linux перед ядром NT.

Системные вызовы делегируются примитивам NT, а не вызовам Win32.
Важно: нет перевода системных вызовов Linux в системные вызовы Win32.

Проблемы WSL1:

- WSL 1 должен реализовать все двоичные интерфейсы приложений (ABI) ядра Linux «бит к биту».

Если в интерфейсе есть ошибка, WSL 1 должен её воспроизвести.
Если есть функция, которая отсутствует в ядре NT, то она либо не может быть реализована, либо нуждается в дополнительной логике ядра (и поэтому становится медленнее).

- Подсистема Linux в WSL 1 должна соблюдать любые «ограничения» и внутренние различия, существующие между ядром NT и традиционным дизайном Unix.

Наиболее очевидным отличием является файловая система NTFS и её семантика, а также то, как эти различия вредят производительности бинарных файлов Linux.

WSL2

Образ ядра Linux в Windows 10 представляет собой легкую виртуальную машину.

Виртуальная машина внутри себя запускает обычное ядро Linux, файловую систему Linux и стандартный сетевой стек Linux. Всё это работает внутри VM.

Сравнение возможностей

Функция	WSL 1	WSL 2
Интеграция Windows и Linux	✅	✅
Быстрый запуск	✅	✅
Незначительный расход ресурсов	✅	✅
Запуск с использованием текущих версий VMware и VirtualBox	✅	✅
Управляемая виртуальная машина	❌	✅
Полнофункциональное ядро Linux	❌	✅
Полная совместимость системных вызовов	❌	✅
Производительность в файловых системах ОС	✅	❌

Wine

- Wine («Wine Is Not an Emulator»)— это программный инструмент для запуска игр и приложений, созданных для платформы Windows, в операционных системах на ядре Linux. В отличие от традиционных эмуляторов, он выполняет преобразование API-вызовов Windows в системные вызовы ядра Linux без использования виртуализации.